
Topic 10

Software Effort Estimation

Effort estimation is the process of predicting the amount of effort required to complete a software project. It involves determining how many resources (such as time, personnel, and money) will be required to achieve the project objectives.

There are various methods for estimating the effort needed for a software project. The most common ones are:

1. **Function Points**
2. **Lines of Code (LOC)**
3. **COCOMO Model (CONstructive COst MODEL)**

1. Function Points

What are Function Points?

Function points are a unit of measurement for software development. They are used to quantify the functionality provided to the user based on the system's requirements. Function points are independent of the technology or programming language used to develop the software.

Function points focus on the software's functional requirements, such as inputs, outputs, data storage, and user interactions.

Steps to Calculate Function Points:

1. **Identify and classify the system's functions:** These functions include inputs (e.g., forms), outputs (e.g., reports), inquiries (e.g., search functions), internal files (e.g., databases), and external interfaces (e.g., communication with other systems).
2. **Assign complexity weights:** Each function is assigned a complexity level: low, average, or high. The complexity determines the weight of each function.
3. **Calculate unadjusted function points:** Sum the weighted complexity of all the identified functions.
4. **Apply a value adjustment factor (VAF):** This factor adjusts the function point count based on 14 general system characteristics such as reliability, data security, etc. Each characteristic is rated on a scale from 0 to 5, and the total score is used to adjust the unadjusted function points.

Example:

Suppose a system has the following:

- 3 external inputs (with an average complexity weight of 4 points each)
- 2 external outputs (with an average complexity weight of 5 points each)

- 1 internal logical file (with a high complexity weight of 7 points)

The unadjusted function points = $(3 \times 4) + (2 \times 5) + (1 \times 7) = 12 + 10 + 7 = 29$ function points.

Let's say the VAF is 1.2 (based on system characteristics). Adjusted function points = $29 \times 1.2 = 34.8$ function points.

Application:

Function points are often used to measure the size of software systems and estimate the effort required for development, maintenance, and testing.

2. Lines of Code (LOC)

What is LOC?

Lines of code is a traditional software effort estimation method where the effort is measured based on the number of lines of code written for the project. This approach assumes that more lines of code result in more effort needed to write, test, and maintain the software.

Steps to Estimate Using LOC:

1. **Estimate the total number of lines of code** that will be needed for the project. This can be done by analyzing the project's requirements or using historical data from similar projects.
2. **Calculate the effort** based on the number of lines of code. This is usually done using a productivity factor, which represents the number of lines of code a developer can produce in a given time period (e.g., lines per day).

Example:

Suppose you estimate that the project will require 10,000 lines of code. If your productivity factor is 50 lines of code per day, the estimated number of developer days required is:

$$10,000 \text{ lines} / 50 \text{ lines/day} = 200 \text{ developer days}$$

Application:

LOC can be a quick estimation method, but it has limitations. It does not consider the quality of code or complexity. Writing more code doesn't necessarily mean higher effort if the code is simple or modular.

3. COCOMO Model (COConstructive COst MOdel)

What is COCOMO?

COCOMO is a widely used model for software effort estimation developed by Barry Boehm in 1981. It uses a mathematical formula to estimate the cost and effort required for a software project based on its size and other factors.

There are three types of COCOMO models:

1. **Basic COCOMO Model:** Provides a rough estimate of effort based on the size of the software in terms of thousands of lines of code (KLOC).
2. **Intermediate COCOMO Model:** Considers factors that affect software development, such as product reliability, database complexity, and programmer experience.
3. **Detailed COCOMO Model:** Provides a more detailed estimate by considering factors at each stage of development (requirements, design, coding, testing, etc.).

Basic COCOMO Formula:

The basic COCOMO formula is:

$$\text{Effort (Person-Months)} = a \times (\text{KLOC})^b$$

Where:

- KLOCKLOC = Estimated size of the software in thousands of lines of code
- aa, bb = Constants based on the type of project (organic, semi-detached, embedded)

For example, for an "organic" project (simple systems), the constants are:

- $a=2.4$
- $b=1.05$

Example:

Suppose you estimate the size of your project to be 50,000 lines of code (i.e., 50 KLOC) for an organic project. Using the formula:

$$\text{Effort} = 2.4 \times (50)^{1.05} = 2.4 \times 52.7 = 126.48 \text{ person-months}$$

This means the project will require approximately 126 person-months of effort.

Intermediate COCOMO Model:

The intermediate COCOMO model refines the estimate by considering factors like:

- Product reliability
- Data complexity
- Personnel experience

Each of these factors is rated on a scale, and the ratings are used to adjust the basic effort estimate.

Application:

COCOMO is widely used for large-scale projects to get a more accurate estimate of the required effort, cost, and time. It also allows for sensitivity analysis by adjusting the model's factors.

Scenario-Based Questions:

Scenario 1: You are managing a project to develop a customer relationship management (CRM) system. Based on the project requirements, you estimate that the system will consist of 8 external inputs, 5 external outputs, 2 internal files, and 4 inquiries. Using the function point method, calculate the unadjusted function points if the complexity weights are as follows:

- External inputs: Average complexity (4 points each)
- External outputs: Average complexity (5 points each)
- Internal files: High complexity (7 points each)
- Inquiries: Low complexity (4 points each)

Solution: Unadjusted function points = $(8 \times 4) + (5 \times 5) + (2 \times 7) + (4 \times 4) = 32 + 25 + 14 + 16 = 87$ function points.

Scenario 2: You are estimating the effort required for a software project, and the estimated size is 12,000 lines of code. The productivity factor for the developers is 40 lines per day. How many developer days will it take to complete the project?

Solution: **Estimated developer days = 12,000 lines / 40 lines/day = 300 developer days**

Scenario 3: You are managing a software project that is expected to have a size of 100 KLOC (thousands of lines of code). The project is classified as "semi-detached" in terms of complexity. Using the COCOMO model, calculate the estimated effort required for the project. The constants for the semi-detached project are:

- $a=3.0$
- $b=1.12$

Solution: **Effort = $3.0 \times (100)^{1.12}$ Effort = 3.0**

This would be the estimated effort for the project.